



UNIVERSITY OF
MAURITIUS

Database Systems Assignment

ICDT 1202Y

BSc. (Hons) Computer Science
Level 1 Semester 2



Govind Dhruv	2510502
Sahajoo Ronitesh	2512131
Makhun Lokeshav Anand	2514204
Khedoo Muhammad Younus Khan Bin Yousouf	2513292
Bungchee Tejasvir	2514012

Table of Contents

1. Detailed System Description	3
1.1 Introduction & Objectives	3
1.2 Users of the System	3
1.3 Core Functional Modules & Business Rules	3
1.4 Scope & Limitations	5
2. Entity Relationship Diagram (ERD)	5
<i>List of entities</i>	5
<i>Cardinalities</i>	5
3. Normalisation	6
<i>Functional dependencies</i>	6
1NF	7
2NF	7
3NF	8
<i>Final List of Normalised Tables</i>	8
<i>Foreign keys</i>	8
<i>Evaluation</i>	8
4. Final Database Design	9
5. Stored Procedures	11
6. Triggers & Cursors	13
7. Security	14
1. Users and Logins	14
2. Roles & Privileges	14
3. Granting and Revoking Privileges	15
4. Access Control using DCL	15
8. Complex Queries	15
1. Most rented vehicle	15
2. Customer rental history	15
3. Late return list	16
9. Testing	16
References	22
Individual Contribution	23

1. Detailed System Description

1.1 Introduction & Objectives

Car Rental companies manage large volume of information such as customer details, car details, transactions. Managing this information manually can be inefficient as it is time-consuming, leading to errors and no proper data organisation. Therefore, the use of a database system is necessary to ensure efficient and accurate data management.

A **Car Rental Database System** will be designed to store, organise, and manage related information. The system will help users to monitor activities such as car availability, customer registration, and reservation amongst others. By centralising all the data in one place, we facilitate manipulation of information, ensure data integrity, and improve overall operations management.

This project will focus on designing and implementing a relational database system for a Car Rental company using SQL Server Management Studio. The system will consist of several tables such as customers, vehicles, and transactions. The use of normalisation process up to 3NF is necessary to minimize redundancy and maintain consistency. The system will also consist of stored procedures, triggers, and security measures to simulate real-world business scenarios.

The aim of this project is to demonstrate an understanding of database design and implementation, as well as the ability to work comfortably with **SQL** syntax and database management concepts.

1.2 Users of the System

The Car Rental Database System is used by different types of users within the organisation. Each user has specific roles and responsibilities in managing and operating the system.

- **Admin:** Responsible for managing the overall system, including maintaining vehicle, customer, and staff records.
- **Employee:** Handles daily rental operations such as registering customers, creating bookings, and recording vehicle rentals and returns.
- **Accountant:** Manages financial records, including rental payments and generating basic financial reports.

1.3 Core Functional Modules & Business Rules

The **Car Rental Database System** is divided into several functional modules that manage various aspects of the rental process. The following table lists the modules and their descriptions including the rules followed by each module.

Modules	Description
1. Customer Management	This module stores and manages customer information such as name, contact details, address, and driver's license number. Each customer is assigned a unique ID to track reservations and rental history. Each customer must have a unique ID, a valid driver's license number (not expired), and complete contact details.

2. Vehicle & Category Management	This module manages vehicle details and organizes them into categories such as Economy, Sedan, and SUV. It also tracks the vehicle status (Available, Rented, or Under Maintenance). Every vehicle must belong to exactly one of three categories: Economy, Sedan, or SUV. Each category must have a defined daily rental rate used for payment calculations. A vehicle's status must always be either Available, Rented or Under Maintenance.
3. Reservation Management	The Reservation Management module allows customers to reserve vehicles in advance. It records reservation details and ensures that vehicles are not double-booked during the selected period. A vehicle cannot be reserved by more than one customer for overlapping date ranges. A reservation must have a valid start date and end date. A reservation can only be made for a vehicle with Available status.
4. Rental Processing	This module manages the actual rental transaction when a customer collects a vehicle. It records the rental period and updates the vehicle status accordingly. A rental can only begin if the vehicle's status is Available or has a confirmed reservation. Vehicle status changes to Rented when rental starts, and back to Available upon return (unless Under Maintenance is triggered). A vehicle cannot be rented to two customers at the same time.
5. Payment Processing	This module records payments for rentals and calculates the total rental cost based on the vehicle category and rental duration. A receipt/payment record must reference a valid, existing rental ID. Total rental cost = Daily rate of the vehicle's category × Number of rental days. Late returns incur additional charges based on extra days. The rental is only considered closed once fully paid.
6. Maintenance Tracking	This module keeps a record of vehicle maintenance and service. Vehicles under maintenance are marked as unavailable for rental. When a vehicle is scheduled for or undergoing maintenance, its status must be set to Under Maintenance (for reservation or rental), with a start date and end date. The vehicle returns to Available status only after maintenance is completed.

7. Reporting & Analytics	This module generates reports such as rental history, vehicle availability, revenue summaries to help monitor business operations. Rental history only includes completed and fully paid rentals while revenue figures are drawn from confirmed payments only. Vehicle availability reflects real-time status.
-------------------------------------	---

1.4 Scope & Limitations

The Car Rental Database system is designed to record and manage data related to car rental transactions in an organised and secure way. It manages customer records, vehicle inventory, reservations, rentals, payments and maintenance. The system is built as a centralised platform for employees of the company to use regarding all core operations of the business. It is run entirely within Microsoft SQL Server by the internal staff only, meaning is not accessible to anyone outside the organisation.

Although the system manages core rental operations, it has several **limitations** as listed below. Limitations include:

- **No different types of payment integration:** Employees must handle payments outside database and then enter records in the database manually.
- **No Real-Time vehicle Tracking:** Real-time location of rentals is not included in the system such as GPS tracking.
- **No mobile application support:** The system is designed for desktop use only and does not provide a mobile application for customers or staff.
- **No online booking interface for customers:** Reservations must be entered manually by staff. (Like all other records)
- **No integration with external systems:** The database operates as a standalone system and cannot connect to external software such as any accounting tool.

2. Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) represents the structure of the Car Rental database system. It illustrates the main entities, their attributes, and the relationships between them. The ERD helps to visualise how data is organised and how different entities interact with the system.

List of entities

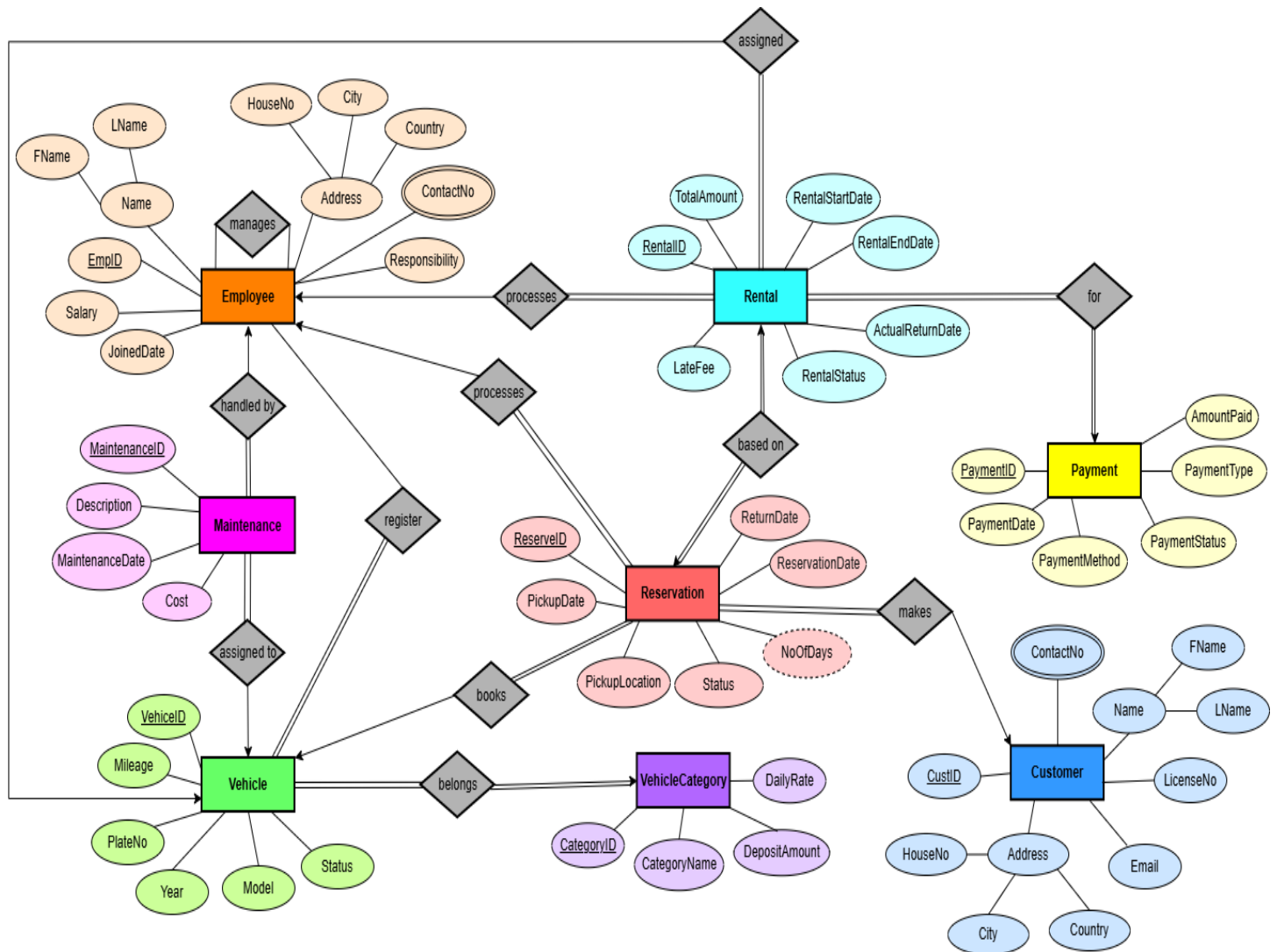
Customer, VehicleCategory, Vehicle, Reservation, Rental, Payment, Employee, Maintenance

Cardinalities

Customer - Reservation	One to Many
Reservation - Rental	One to One
Rental - Payment	One to Many
VehicleCategory - Vehicle	One to Many
Vehicle - Reservation	One to Many

Vehicle - Rental	One to Many
Vehicle - Maintenance	One to Many
Employee - Rental	One to Many
Employee - Reservation	One to Many
Employee - Maintenance	One to Many

The above relationships represent the cardinality between entities in the car rental system. The **ERD** below represents the structure of the Car Rental Database System.



3. Normalisation

Functional dependencies

The functional dependencies identified are listed below.

CustID → CustFname, CustLname, CustContactNo, LicenseNo, Email, CustHouseNo, CustCity, CustCountry

ReserveID → ReservationDate, PickupDate, ReturnDate, PickupLocation, ReservationStatus, CustID, VehicleID, EmpID

RentalID → RentalStartDate, RentalEndDate, ActualReturnDate, RentalStatus, TotalAmount, LateFee, ReserveID, CustID, VehicleID, EmpID

PaymentID → PaymentDate, PaymentMethod, PaymentStatus, PaymentType, AmountPaid, RentalID, ReserveID

VehicleID → PlateNo, Model, Year, Mileage, Status, CategoryID

CategoryID → CategoryName, DailyRate, DepositAmount

MaintenanceID → MaintenanceDate, Description, Cost, VehicleID, EmpID

EmpID → EmpFname, EmpLname, EmpContactNo, Salary, JoinedDate, Responsibility, EmpHouseNo, EmpCity, EmpCountry

ONF

- In un-normalised form (0 NF or UNF), all the data related to car rental system is stored onto a single table.

CarRental {CustID, CustName, CustContactNo, LicenseNo, Email, CustHouseNo, CustCity, CustCountry, VehicleID, PlateNo, Model, Year, Mileage, Status, CategoryID, CategoryName, DailyRate, DepositAmount, ReserveID, ReservationDate, PickupDate, ReturnDate, PickupLocation, ReservationStatus, RentalID, RentalStartDate, RentalEndDate, ActualReturnDate, RentalStatus, TotalAmount, LateFee, PaymentID, PaymentDate, PaymentMethod, PaymentStatus, PaymentType, AmountPaid, EmpID, Empname, EmpContactNo, Salary, JoinedDate, Responsibility, EmpAddress, MaintenanceID, MaintenanceDate, Description, Cost}

The table contains repeating groups and multiple values for some attributes

Issues with ONF: Repeating groups of data, redundancy and possibility of insert, delete or update anomalies.

1NF

- Identify primary key
- Eliminate repeating groups
- Fields containing atomic value

CarRental {CustID, CustFname, CustLname, CustContactNo, LicenseNo, Email, CustHouseNo, CustCity, CustCountry, VehicleID, PlateNo, Model, Year, Mileage, Status, CategoryID, CategoryName, DailyRate, DepositAmount, ReserveID, ReservationDate, PickupDate, ReturnDate, PickupLocation, ReservationStatus, RentalID, RentalStartDate, RentalEndDate, ActualReturnDate, RentalStatus, TotalAmount, LateFee, PaymentID, PaymentDate, PaymentMethod, PaymentStatus, PaymentType, AmountPaid, EmpID, EmpFname, EmpLname, EmpContactNo, Salary, JoinedDate, Responsibility, EmpHouseNo, EmpCity, EmpCountry, MaintenanceID, MaintenanceDate, Description, Cost}

2NF

- No partial dependencies (Depend on whole composite key, not part of it)

Customer { CustID, CustFname, CustLname, CustContactNo, LicenseNo, Email, CustHouseNo, CustCity, CustCountry }

Reservation { ReserveID, ReservationDate, PickupDate, ReturnDate, PickupLocation, ReservationStatus, CustID, VehicleID, EmpID }

Rental { RentalID, RentalStartDate, RentalEndDate, ActualReturnDate, RentalStatus, TotalAmount, LateFee, ReserveID, CustID, VehicleID, EmpID }

Payment { PaymentID, PaymentDate, PaymentMethod, PaymentStatus, PaymentType, AmountPaid, RentalID, ReserveID }

Vehicle { VehicleID, PlateNo, Model, Year, Mileage, Status, CategoryID }

VehicleCategory { CategoryID, CategoryName, DailyRate, DepositAmount }

Maintenance { MaintenanceID, MaintenanceDate, Description, Cost, VehicleID, EmpID }

Employee { EmpID, EmpFname, EmpLname, EmpContactNo, Salary, JoinedDate, Responsibility, EmpHouseNo, EmpCity, EmpCountry }

3NF

- No transitive dependencies (non-key attribute depending on other non-key attribute)

Customer { CustID, CustFname, CustLname, CustContactNo, LicenseNo, Email, CustHouseNo, CustCity, CustCountry }

Reservation { ReserveID, ReservationDate, PickupDate, ReturnDate, PickupLocation, ReservationStatus, CustID, VehicleID, EmpID }

Rental { RentalID, RentalStartDate, RentalEndDate, ActualReturnDate, RentalStatus, TotalAmount, LateFee, ReserveID, CustID, VehicleID, EmpID }

Payment { PaymentID, PaymentDate, PaymentMethod, PaymentStatus, PaymentType, AmountPaid, RentalID, ReserveID }

Vehicle { VehicleID, PlateNo, Model, Year, Mileage, Status, CategoryID }

VehicleCategory { CategoryID, CategoryName, DailyRate, DepositAmount }

Maintenance { MaintenanceID, MaintenanceDate, Description, Cost, VehicleID , EmpID }

Employee { EmpID, EmpFname , EmpLname, EmpContactNo, Salary, JoinedDate, Responsibility, EmpHouseNo, EmpCity, EmpCountry }

Final List of Normalised Tables

After applying normalisation, the database consists of the following tables:

Customer, VehicleCategory, Vehicle, Reservation, Rental, Payment, Employee, Maintenance

Foreign keys

Reservation [FK CustID → Customer(CustID) , FK VehicleID → Vehicle(VehicleID) , FK EmpID → Employee(EmpID)]

Rental [FK ReserveID → Reservation(ReserveID) , FK CustID → Customer(CustID) , FK VehicleID → Vehicle(VehicleID) , FK EmpID → Employee(EmpID)]

Payment [FK RentalID → Rental(RentalID), FK ReserveID→ Reservation(ReserveID)]

Vehicle [FK CategoryID → VehicleCategory(CategoryID)]

Maintenance [FK VehicleID → Vehicle(VehicleID) , FK EmpID → Employee(EmpID)]

Evaluation

- We observed that there is no difference between 2NF and 3NF, this is because while removing partial dependencies there were no remaining transitive dependencies and therefore the normalisation process up to 3NF was completed.
- We also observed that reservation and rental both has foreign key [Customer(CustID), Vehicle(VehicleID), Employee(EmpID)], this is because there could be a vehicle rent without reservation.
- After normalisation from 0NF to 3NF, the database system has no data redundancy and all non-key attributes depend on corresponding primary key, this results in improved data integrity and reduces anomalies.

4. Final Database Design

The database was implemented using **Microsoft SQL Server**. The following tables present the structure of each entity's field names, data types and constraints. These definitions ensure data integrity and consistency within the database.

1. Customer Table

Field Name	Data Type	Constraints
CustID	VARCHAR(10)	PRIMARY KEY, Starts with 'C'
CustFname	VARCHAR(50)	NOT NULL
CustLname	VARCHAR(50)	NOT NULL
CustContactNo	VARCHAR(15)	NOT NULL, UNIQUE
LicenseNo	VARCHAR(20)	NOT NULL, UNIQUE
Email	VARCHAR(75)	NOT NULL, UNIQUE, Should contain '@' and '.'
CustHouseNo	VARCHAR(20)	NOT NULL
CustCity	VARCHAR(50)	NOT NULL
CustCountry	VARCHAR(50)	NOT NULL

2. VehicleCategory Table

Field Name	Data Type	Constraints
CategoryID	CHAR(3)	PRIMARY KEY
CategoryName	VARCHAR(50)	NOT NULL, Possible Values: 'ECO', 'SDN', 'SUV'
DailyRate	DECIMAL(10, 2)	NOT NULL, Should be ≥ 0
DepositAmount	DECIMAL(10, 2)	NOT NULL, Should be ≥ 0

3. Vehicle Table

Field Name	Data Type	Constraints
VehicleID	CHAR(10)	PRIMARY KEY
PlateNo	VARCHAR(20)	NOT NULL, UNIQUE
Model	VARCHAR(100)	NOT NULL
VYear	INTEGR	NOT NULL, Should be ≥ 1900 and $<$ current year
Mileage	INTEGER	NOT NULL, Default as 0, Should be > 0
VehicleStatus	VARCHAR(20)	NOT NULL, Default as 'Available', Possible Values: 'Available', 'Rented', 'Under Maintenance'
CategoryID	CHAR(3)	NOT NULL, Foreign Key references VehicleCategory

4. Employee Table

Field Name	Data Type	Constraints
EmpID	INTEGER	PRIMARY KEY
EmpFname	VARCHAR(50)	NOT NULL
EmpLname	VARCHAR(50)	NOT NULL
EmpContactNo	VARCHAR(15)	NOT NULL, UNIQUE
Salary	DECIMAL(10, 2)	Should be ≥ 0
JoinedDate	DATE	NOT NULL
Responsibility	VARCHAR(100)	
EmpHouseNo	VARCHAR(20)	

EmpCity	VARCHAR(50)	
EmpCountry	VARCHAR(50)	Default as 'Mauritius'

5. Reservation Table

Field Name	Data Type	Constraints
ReserveID	VARCHAR(10)	PRIMARY KEY, Starts with 'R'
ReservationDate	DATE	NOT NULL
PickupDate	DATE	NOT NULL
ReturnDate	DATE	NOT NULL
PickupLocation	VARCHAR(100)	NOT NULL
ReservationStatus	VARCHAR(20)	NOT NULL, Default as 'Pending'
CustID	VARCHAR(10)	Starts with 'C', Foreign Key references Customer
VehicleID	CHAR(10)	Foreign Key references Vehicle
EmpID	INTEGER	Foreign Key references Employee

6. Rental Table

Field Name	Data Type	Constraints
RentalID	VARCHAR(10)	PRIMARY KEY, Starts with 'rid'
RentalStartDate	DATE	NOT NULL
RentalEndDate	DATE	NOT NULL
ActualReturnDate	DATE	
RentalStatus	VARCHAR(20)	NOT NULL, Default as 'Ongoing', Possible Values: 'Ongoing', 'Completed', 'Cancelled'
TotalAmount	DECIMAL(10, 2)	NOT NULL
LateFee	DECIMAL(6, 2)	
ReserveID	VARCHAR(10)	Starts with 'R', Foreign Key references Reservation
CustID	VARCHAR(10)	Starts with 'C', Foreign Key references Customer
VehicleID	CHAR(10)	Foreign Key references Vehicle
EmpID	INTEGER	Foreign Key references Employee

7. Payment Table

Field Name	Data Type	Constraints
PaymentID	CHAR(3)	PRIMARY KEY, Starts with 'P'
PaymentDate	DATE	NOT NULL
PaymentMethod	VARCHAR(10)	NOT NULL, Possible values: 'Cash', 'Card', 'Online'
PaymentStatus	VARCHAR(20)	NOT NULL, Default as 'Pending', Possible Values: 'Pending', 'Completed', 'Failed'
PaymentType	VARCHAR(20)	NOT NULL, Possible Values: 'Deposit', 'Rental'
AmountPaid	DECIMAL(10, 2)	NOT NULL, Should be > 0
RentalID	VARCHAR(10)	Foreign Key references Rental
ReserveID	VARCHAR(10)	Foreign Key references Reservation

8. Maintenance Table

Field Name	Data Type	Constraints
MaintenanceID	CHAR(5)	PRIMARY KEY, Starts with 'M', NOT NULL
MaintenanceDate	DATE	NOT NULL
Description	VARCHAR(40)	
Cost	DECIMAL(10, 2)	Should be >= 0
VehicleID	CHAR(10)	NOT NULL, Foreign Key references Vehicle
EmpID	INTEGER	NOT NULL, Foreign Key references Employee

5. Stored Procedures

Stored procedures were implemented to **automate key operations** of the car rental management system. These procedures encapsulate business logic within the database, allowing efficient execution of tasks such as customer registration, vehicle management, reservation processing, rental cost calculation and reporting. The use of stored procedures improves performance, ensures consistency and enhances database security. The stored procedures used are described below.

1. sp_add_customer

This stored procedure is used to register a new customer in the system by inserting their personal and contact details into the Customer table. It ensures that all required fields are provided and respects constraints such as unique contact number, license number, and valid customer ID format. Error handling is included to manage any violations or invalid data during insertion.

2. sp_add_vehicle

This stored procedure is used to add the details of a new vehicle to the system by inserting **VehicleID, PlateNo, Model, VYear, Mileage, Status, CategoryID** in the **Vehicle** Table. If the insert is successful, 'Vehicle added successfully' message will be displayed. While doing this procedure, it also handles errors that could possibly happen and display an appropriate message.

3. sp_make_reservation

The '**sp_make_reservation**' stored procedure allows a customer to reserve an available vehicle for a specified rental period. It accepts the reservation details such as the **CustID, VehicleID, ReserveID, EmpID, PickupDate, ReturnDate, PickupLocation** as input parameters. Before inserting the reservation, it performs a series of validations to ensure that the customer, vehicle, and employee exist in the system, that the vehicle is available, that the dates are valid, and that the vehicle is not already reserved for overlapping dates. If all validations **pass**, a new reservation record is inserted with a **default status** of **Pending**. If any validation **fails**, an error is raised and the insertion is cancelled.

4. sp_calculate_rental_cost

This stored procedure calculates the total cost of a rental for a given reservation. From the **RentalID** and **ActualReturnDate** as input parameters, it retrieves the vehicle's category daily rate and computes the total rental amount based on the number of days between the pickup date (RentalStartDate) and return date (RentalEndDate). If the vehicle is returned **late**, an additional late fee is calculated based on the number of extra days multiplied by the daily rate and applied to the final amount.

5. **sp_convert_reservation_to_rental**

This stored procedure converts an existing reservation into a rental record by retrieving the relevant reservation details and inserting them into the Rental table. It validates the existence of the reservation, then calls the '**sp_calculate_rental_cost**' stored procedure to compute the total rental cost and any applicable late fees. The calculated values are updated in the rental record, and the reservation status is set to *Completed*. Error handling is implemented to ensure any issues are properly reported.

6. **sp_apply_late_fee**

This stored procedure takes **RentalID** as input parameter and checks whether that rented vehicle has been returned after the scheduled return date or on time. If it is returned late then the procedure calculates the **additional fee** to be paid (with a 50% penalty per additional day). The procedure then **updates** the Rental table with the new late fee data.

7. **sp_check_vehicle_availability**

This stored procedure displays whether a requested vehicle is available for rent by inserting the **VehicleID** and based on its status. If the vehicle is **not found** in Vehicle Table, the following message will be displayed: 'VehicleID not found'. Otherwise, the corresponding vehicle's status will be displayed.

8. **sp_make_payment**

This stored procedure processes payments for both reservation deposits and rental payments. By validating payment/deposit details, the payment table is then updated accordingly. For deposits payment, the procedure also updates the reservation status to **Confirmed** in the reservation table. For rental payments, the stored procedure updates the rental status to **Completed** in the rental table after successful transactions. The procedure also prevents duplicate payments and ensures only valid payment methods are used.

9. **sp_monthly_revenue_report**

This stored procedure generates a summary report of the total revenue collected from completed rentals within a specified month and year. It accepts a month (**@Month**) and year (**@Year**) as input parameters and returns key figures such as the total number of rentals, total revenue, average revenue per rental, and the highest and lowest single rental amounts for that period.

10. **sp_vehicle_returned**

The '**sp_vehicle_returned**' stored procedure updates the rental record when a vehicle is returned. After proper validations and verifications, the procedure records the actual return date in the Rental table. This update can trigger related processes, such as calculating late fees and updating vehicle availability status through associated triggers. Hence, accurate tracking of returned vehicles and proper rental closure management are ensured.

6. Triggers & Cursors

Triggers

Triggers are special stored procedures that run automatically when an **INSERT**, **UPDATE** or **DELETE** operation is performed on a table. These procedures are essential to automate actions and keep data consistent such as updating vehicle status, fees, rental status, payment and reservation while enforcing business rules. The triggers used are listed and described below.

1. tg_update_vehicle_status

This trigger will update the **VehicleStatus** in **Vehicle Table** based on changes in the rental record. If a **new rental** has been **added**, the trigger will check its **rental_status** field for the value **'Ongoing'**. As per the above condition, the Vehicle Status will be set to **'Rented'** since it is no longer available. However, if the rental's status has changed to **'Completed'**, the vehicle status will be reset to **'Available'**. Conditions; such as **<> 'Rented'** and **<> 'Available'**; prevent updating the VehicleStatus field unnecessarily if it is already updated appropriately.

2. tg_overlapping_reservation

This trigger is activated when there is an insert in the reservation table such that for same vehicle the renting periods are overlapping then this error message is displayed 'This vehicle is already booked for this period'. First, the insertion is allowed, then it checks for the same **VehicleID** where the **ReserveID** is not equal to the inserted one. If there is any overlapping, the table **ROLLBACK** to the state it was before the insert.

3. tg_auto_late_fee

This trigger is activated whenever an **update** is made in the **Rental** table in the **ActualReturnDate** column. It retrieves the **ActualReturnDate** and **RentalEndDate** and determines whether there is a late fee to be paid or not (fee is set to 0 if return date is earlier or on the same date as the scheduled Return date).

4. tg_update_payment_status

This trigger automatically updates the payment status (Field: **PaymentStatus**) in the **Payment** table to **'Completed'** when a payment is made. It is triggered after a payment is done for the first time or when it is updated hence the use of the **'AFTER INSERT, UPDATE'** trigger.

The trigger also contains a **TRY...CATCH** block that handles any runtime errors that may happen during execution. In case of any error, the type of error is identified (**ERROR_NUMBER()**) and a descriptive message of what went wrong is provided (**ERROR_MESSAGE()**). This ensures that the system does not fail silently and facilitates debugging.

Cursors

A **cursor** is used to process data one row at a time (row-by-row handling), rather than all at once as a set (bulk processing). It temporarily stores data for operations like **SELECT**, **UPDATE** or **DELETE**. In the car rental system, cursors are used to identify overdue rentals and to find vehicles that have been returned. The cursors used are described below.

1. Overdue_rental_cursor

This cursor identifies all rentals in the **Rental** table where the vehicle was returned after the **scheduled end date** (i.e. 'ActualReturnDate > RentalEndDate'). It iterates through each overdue rentals and for each record which meets the conditions, a late fee is applied by executing the stored procedure '**sp_apply_late_fee**' with the required **RentalID** as parameter (**@RentalID**).

2. Update_vehicle_status_cursor

Although in practice a set-based approach would be more efficient, for the assignment purpose, a cursor was implemented to iterate through the rental table and store the **VehicleID** in the **@VehicleID** variable where vehicle has been returned and then update the status to 'Available' in the **vehicle** table where **VehicleID** matches **@VehicleID**.

7. Security

Security measures for the Car Rental Database System were implemented using **Data Control Language (DCL)**. The focal point is to control access to data and ensure only people with authorised access are allowed to perform certain operations on the database. This is achieved by creating database users (i.e. Admin, Employee and Accountant) and assigning **passwords** to each role with appropriate privileges. Through this process, integrity of data is protected and unauthorised modifications to the system are prevented.

1. Users and Logins

To provide every person who are involved in the Rentel Service with a unique username and password, we should create respective logins for them. This operation includes **CREATE LOGIN, WITH PASSWORD**. Then for every logins we will create a user using **CREATE USER, FOR LOGIN**. This ensures that only authorised users can have access to the database.

2. Roles & Privileges

The **3 main roles** of the system and their associated privileges are described below.

- **Admin**

Admin will be granted permission to **SELECT, INSERT, UPDATE OR DELETE** on every table present in the database. This ensures that administration of the Car Rental database system is **efficient** and overall **system integrity**. Moreover, the Admin is responsible for user access ensuring that appropriate permissions are granted to **Employees** and **Accountants**.

- **Employee**

The Employee is granted limited access to the database compared to the Admin. This is done to ensure that the employee does not manipulate data such as deleting specific records or even have access to other employees' salaries without authorisation. This enforces data security and integrity in the database system. Employees are granted:

- **SELECT** and **INSERT** privileges on the **Reservation** table to create and view bookings.
- **SELECT** and **UPDATE** privileges on the **Vehicle** table to manage vehicle status.
- **SELECT** privileges on **Customer** and **Rental** tables to access necessary information.

- **Accountant**

The **Accountant** is granted specific access to financial and transaction-related data within the Car Rental database. This ensures that **proper financial oversight** is carried out by the Accountant role while maintaining data security by restricting access and changes to tables that are not relevant to accounting functions.

3. Granting and Revoking Privileges

Proper management of user privileges is essential to maintain database security and operational efficiency. In this system, the **GRANT** command is used to assign specific permissions such as **SELECT, INSERT, UPDATE, DELETE, and EXECUTE** to roles based on their job responsibilities. For example, Accountants are granted execution rights on financial stored procedures, while Employees are given limited data access needed for daily operations. If a user's role changes or access is no longer required, privileges can be modified or removed using the **REVOKE** or **DENY** commands. This flexibility ensures that access rights remain up to date, reduces the risk of unauthorised actions, and helps maintain overall database integrity and security.

4. Access Control using DCL

Access control has been implemented using **DCL** commands such as **GRANT** and **DENY** to manage user permissions. **GRANT** assigns privileges to roles (Admin, Employee and Accountant) while **DENY** explicitly restricts certain actions. Different roles are given appropriate access based on their responsibilities in the database system ensuring sensitive data is not accessible to certain roles like Employee. The system ensures that the users have minimum access required, which reduces the risk of unauthorised access and enforces data integrity.

8. Complex Queries

Complex queries section will demonstrate a deeper understanding of **Data Management Language (DML)** where queries such as **GROUP BY, JOIN, SUM, COUNT OR ORDER BY** will be used to perform operations.

1. Most rented vehicle

This query looks for the most rented vehicles. It takes data from the **Rental, Vehicle and VehicleCategory tables**, grouping everything **by vehicle** so it can count rentals and calculate the total revenue. Any cancelled rentals are left out and after calculations, the results come out ordered from the most rented vehicle to the least, to see which vehicles customers tend to rent the most. This is useful to give the customers the best-rented vehicle in the system to facilitate their choice.

2. Customer rental history

This query retrieves a **detailed rental history** for **each customer**. It combines the data on the **Customer** table and the **Rental** table by using the field '**CustID**' to join the tables. It displays the customer's personal details, the total number of rentals made by the customer across all their rentals, total amount of money spent by each customer across all their rentals, the average cost per

rental for each customer, the highest amount paid by the customer for a single rental and it counts the number of rentals where the vehicle was return late (using a subquery).

Only the customers whose total spending exceeds the average rental amount across all rentals are included in the results (filtered using the **HAVING** clause) and the output is sorted in **Descending order of 'TotalSpent'**, so that the customers who spent the most appear at the top.

3. Late return list

The late return list operation will provide the user with **details about penalties or situations where financial tracking should be monitored**. The list will be ordered in descending order with respect to the actual return date. It includes rental details, customer information, vehicle details, number of late days, and applicable late fees.

9. Testing

The testing section of this report is to ensure that the **Car Rental Database System** functions correctly and securely. This process helps to identify errors and make sure every feature (triggers, security, stored procedures and SQL queries) work as expected and data security is enforced.

1. Setup Initial Data

Testing Script

```
-- Insert Categories
INSERT INTO VehicleCategory (CategoryID, CategoryName, DailyRate, DepositAmount) VALUES
('C01', 'ECO', 1000.00, 5000.00),
('C02', 'SDN', 1500.00, 7000.00),
('C03', 'SUV', 2500.00, 10000.00);

-- Insert Employees
INSERT INTO Employee (EmpID, EmpFname, EmpLname, EmpContactNo, Salary, JoinedDate, Responsibility) VALUES
(101, 'John', 'Doe', '51234567', 25000, GETDATE(), 'Manager'),
(102, 'Jane', 'Smith', '57654321', 20000, GETDATE(), 'Clerk');

--Verify
SELECT * FROM VehicleCategory
SELECT * FROM Employee
```

VehicleCategory Table

	CategoryID	CategoryName	DailyRate	DepositAmount
1	C01	ECO	1000.00	5000.00
2	C02	SDN	1500.00	7000.00
3	C03	SUV	2500.00	10000.00

Employee Table

	EmpID	EmpFname	EmpLname	EmpContactNo	Salary	JoinedDate	Responsibility	EmpHouseNo	EmpCity	EmpCountry
1	101	John	Doe	51234567	25000.00	2026-04-05	Manager	NULL	NULL	Mauritius
2	102	Jane	Smith	57654321	20000.00	2026-04-05	Clerk	NULL	NULL	Mauritius

2. Adding 10 customers

Testing Script

```
EXEC sp_add_customer 'C001', 'Alice', 'Brown', '58010001', 'LIC001', 'alice@test.com', '10', 'Port Louis', 'Mauritius';
EXEC sp_add_customer 'C002', 'Bob', 'White', '58010002', 'LIC002', 'bob@test.com', '12', 'Curepipe', 'Mauritius';
EXEC sp_add_customer 'C003', 'Charlie', 'Black', '58010003', 'LIC003', 'charlie@test.com', '5', 'Vacoas', 'Mauritius';
EXEC sp_add_customer 'C004', 'David', 'Green', '58010004', 'LIC004', 'david@test.com', '21', 'Quatre Bornes', 'Mauritius';
EXEC sp_add_customer 'C005', 'Eve', 'Blue', '58010005', 'LIC005', 'eve@test.com', '9', 'Rose Hill', 'Mauritius';
EXEC sp_add_customer 'C006', 'Frank', 'Grey', '58010006', 'LIC006', 'frank@test.com', '3', 'Moka', 'Mauritius';
EXEC sp_add_customer 'C007', 'Grace', 'Yellow', '58010007', 'LIC007', 'grace@test.com', '14', 'Flic en Flac', 'Mauritius';
EXEC sp_add_customer 'C008', 'Hank', 'Red', '58010008', 'LIC008', 'hank@test.com', '8', 'Tamarin', 'Mauritius';
EXEC sp_add_customer 'C009', 'Ivy', 'Pink', '58010009', 'LIC009', 'ivy@test.com', '2', 'Mahebourg', 'Mauritius';
EXEC sp_add_customer 'C010', 'Jack', 'Orange', '58010010', 'LIC010', 'jack@test.com', '11', 'Grand Baie', 'Mauritius';

-- Verify
SELECT * FROM Customer;
```

Customer Table

	CustID	CustFname	CustLname	CustContactNo	LicenseNo	Email	CustHouseNo	CustCity	CustCountry
1	C001	Alice	Brown	58010001	LIC001	alice@test.com	10	Port Louis	Mauritius
2	C002	Bob	White	58010002	LIC002	bob@test.com	12	Curepipe	Mauritius
3	C003	Charlie	Black	58010003	LIC003	charlie@test.com	5	Vacoas	Mauritius
4	C004	David	Green	58010004	LIC004	david@test.com	21	Quatre Bornes	Mauritius
5	C005	Eve	Blue	58010005	LIC005	eve@test.com	9	Rose Hill	Mauritius
6	C006	Frank	Grey	58010006	LIC006	frank@test.com	3	Moka	Mauritius
7	C007	Grace	Yellow	58010007	LIC007	grace@test.com	14	Flic en Flac	Mauritius
8	C008	Hank	Red	58010008	LIC008	hank@test.com	8	Tamarin	Mauritius
9	C009	Ivy	Pink	58010009	LIC009	ivy@test.com	2	Mahebourg	Mauritius
10	C010	Jack	Orange	58010010	LIC010	jack@test.com	11	Grand Baie	Mauritius

3. Adding 5 Vehicles

Testing Script

```
EXEC sp_add_vehicle 'V001', '1234 ZY 22', 'Toyota Vitz', 2022, 15000, 'Available', 'C01';
EXEC sp_add_vehicle 'V002', '5678 AG 21', 'Honda Civic', 2021, 20000, 'Available', 'C02';
EXEC sp_add_vehicle 'V003', '9012 FB 23', 'Kia Sportage', 2023, 5000, 'Available', 'C03';
EXEC sp_add_vehicle 'V004', '3456 DC 20', 'Suzuki Swift', 2020, 45000, 'Available', 'C01';
EXEC sp_add_vehicle 'V005', '7890 MR 22', 'Hyundai Tucson', 2022, 12000, 'Available', 'C03';

-- Verify
SELECT * FROM Vehicle;
```

Vehicle Table

	VehicleID	PlateNo	Model	VYear	Mileage	VehicleStatus	CategoryID
1	V001	1234 ZY 22	Toyota Vitz	2022	15000	Available	C01
2	V002	5678 AG 21	Honda Civic	2021	20000	Available	C02
3	V003	9012 FB 23	Kia Sportage	2023	5000	Available	C03
4	V004	3456 DC 20	Suzuki Swift	2020	45000	Available	C01
5	V005	7890 MR 22	Hyundai Tucson	2022	12000	Available	C03

4. Adding Reservations

Testing Script

```
-- Successful Reservations
EXEC sp_make_reservation 'R001', 'C001', 'V001', 102, '2026-05-01', '2026-05-05', 'Airport';
EXEC sp_make_reservation 'R002', 'C002', 'V002', 102, '2026-05-10', '2026-05-15', 'Hotel';
EXEC sp_make_reservation 'R003', 'C003', 'V003', 101, '2026-06-01', '2026-06-10', 'Office';

EXEC sp_make_reservation 'R004', 'C005', 'V002', 102, '2026-05-17', '2026-05-20', 'Office';
--Using same vehicle as 'R002' but on different dates (Should Pass)

-- TRIGGER TEST: Overlapping (Should Fail)
-- This should throw: "This vehicle is already booked for this period"
EXEC sp_make_reservation 'R005', 'C004', 'V001', 102, '2026-05-02', '2026-05-04', 'Airport';

SELECT *
FROM Reservation;
```

Result

Messages	Deposit Amount for reservation : R001 is Rs5000.00
Messages	Deposit Amount for reservation : R002 is Rs7000.00
Messages	Deposit Amount for reservation : R003 is Rs10000.00
Messages	Deposit Amount for reservation : R004 is Rs7000.00
Messages	Error caught: Vehicle unavailable for selected period.

Note: This error means that `tg_overlapping_reservation` has been successfully triggered from executing an overlapping reservation.

Reservation Table

	ReserveID	ReservationDate	PickupDate	ReturnDate	PickupLocation	ReservationStatus	CustID	VehicleID	EmpID
1	R001	2026-04-09	2026-05-01	2026-05-05	Airport	Pending	C001	V001	102
2	R002	2026-04-09	2026-05-10	2026-05-15	Hotel	Pending	C002	V002	102
3	R003	2026-04-09	2026-06-01	2026-06-10	Office	Pending	C003	V003	101
4	R004	2026-04-09	2026-05-17	2026-05-20	Office	Pending	C005	V002	102

5. Paying Deposit to confirm reservation

Testing Script

```
EXEC sp_make_payment 'P01', 'R001', NULL, 'Deposit', 10000.00, 'Card'; --Wrong Amount (Should Fail)

--Successful Payments
EXEC sp_make_payment 'P01', 'R001', NULL, 'Deposit', 5000.00, 'Card';
EXEC sp_make_payment 'P02', 'R002', NULL, 'Deposit', 7000.00, 'Card';

SELECT *
FROM Reservation;

SELECT *
FROM Payment;
```

Result

Results		Messages		
ErrorNumber	ErrorMessage	ErrorSeverity	ErrorState	
1	50010	Amount paid does not match required deposit amount.	16	1

Messages
Deposit payment processed successfully. Reservation confirmed.

Messages
Deposit payment processed successfully. Reservation confirmed.

Reservation Table

	ReserveID	ReservationDate	PickupDate	ReturnDate	PickupLocation	ReservationStatus	CustID	VehicleID	EmpID
1	R001	2026-04-09	2026-05-01	2026-05-05	Airport	Confirmed	C001	V001	102
2	R002	2026-04-09	2026-05-10	2026-05-15	Hotel	Confirmed	C002	V002	102
3	R003	2026-04-09	2026-06-01	2026-06-10	Office	Pending	C003	V003	101
4	R004	2026-04-09	2026-05-17	2026-05-20	Office	Pending	C005	V002	102

Payment Table

	PaymentID	PaymentDate	PaymentMethod	PaymentStatus	PaymentType	AmountPaid	RentalID	ReserveID
1	P01	2026-04-09	Card	Completed	Deposit	5000.00	NULL	R001
2	P02	2026-04-09	Card	Completed	Deposit	7000.00	NULL	R002

6. Adding rentals

Converting reservations into rentals:

Testing Script

```
-- Convert R001 to Rental
EXEC sp_convert_reservation_to_rental 'rid001', 'R001';

-- Convert R002 to Rental
EXEC sp_convert_reservation_to_rental 'rid002', 'R002';

-- Convert a reservation to rental without paying deposit
EXEC sp_convert_reservation_to_rental 'rid003', 'R003';

SELECT *
FROM Rental;
```

Result

Messages	
Reservation successfully converted to Rental	

Messages	
Reservation successfully converted to Rental	

Results		Messages		
ErrorNumber	ErrorMessage			
1	50000	Reservation cannot be converted. Must pay Deposit first.		

Note: `sp_ApplyLateFee` and `tg_auto_late_fee` are being tested and work correctly. The late fee has been applied to `rid002` as the `ActualReturnDate` is past the `ReturnEndDate`. This also shows that the trigger has successfully executed on updating the rental table.

Rental Table

	RentalID	RentalStartDate	RentalEndDate	ActualReturnDate	RentalStatus	TotalAmount	LateFee	ReserveID	CustID	VehicleID	EmpID
1	rd001	2026-05-01	2026-05-05	NULL	Ongoing	4000.00	0.00	R001	C001	V001	102
2	rd002	2026-05-10	2026-05-15	NULL	Ongoing	7500.00	0.00	R002	C002	V002	102

7. Returning Vehicle, Calculating rental cost and Payment

Testing Script

```
--Return On Time
EXEC sp_vehicle_returned 'rid001', '2026-05-05';
EXEC sp_calculate_rental_cost 'rid001', '2026-05-05';
EXEC sp_make_payment 'P03', NULL, 'rid001', 'Rental', 4000.00, 'Card';
```

Result for
Return On time

Messages						
Vehicle returned. Rental table has been updated.						
Results						
	TotalRentalCost	DaysRented	LateDays	DailyRate	NormalCharge	LateFee
1	4000.00	4	0	1000.00	4000.00	0.000

Payment successful

Messages						
Rental payment processed successfully. Rental completed.						

Testing Script

```
--(Late Return to test Late Fee Trigger)
-- Note: Rental End was 2026-05-15, we return it on 2026-05-17
EXEC sp_vehicle_returned 'rid002', '2026-05-17';
EXEC sp_calculate_rental_cost 'rid002', '2026-05-17';
EXEC sp_make_payment 'P04', NULL, 'rid002', 'Rental', 12000.00, 'Cash';
```

Result for
Late Return to test
Late fee Trigger

2 days late

Messages						
Vehicle returned. Rental table has been updated.						
Results						
	TotalRentalCost	DaysRented	LateDays	DailyRate	NormalCharge	LateFee
1	12000.00	5	2	1500.00	7500.00	4500.000

Payment successful

Messages						
Rental payment processed successfully. Rental completed.						

Testing Script to see
Rental and Payment
Tables

```
SELECT *
FROM Rental;

SELECT *
FROM Payment;
```

Rental Table

	RentalID	RentalStartDate	RentalEndDate	ActualReturnDate	RentalStatus	TotalAmount	LateFee	ReserveID	CustID	VehicleID	EmpID
1	rid001	2026-05-01	2026-05-05	2026-05-05	Completed	4000.00	0.00	R001	C001	V001	102
2	rid002	2026-05-10	2026-05-15	2026-05-17	Completed	12000.00	4500.00	R002	C002	V002	102

Payment Table

	PaymentID	PaymentDate	PaymentMethod	PaymentStatus	PaymentType	AmountPaid	RentalID	ReserveID
1	P01	2026-04-09	Card	Completed	Deposit	5000.00	NULL	R001
2	P02	2026-04-09	Card	Completed	Deposit	7000.00	NULL	R002
3	P03	2026-04-09	Card	Completed	Rental	4000.00	rid001	NULL
4	P04	2026-04-09	Cash	Completed	Rental	12000.00	rid002	NULL

8. Generating monthly revenue report

Testing Script

```
--Check if the costs match
EXEC sp_monthly_revenue_report 4 , 2026;

SELECT *
FROM Payment;
```

Result

ReportMonth	ReportYear	TotalRentals	TotalRentalRevenue	AverageRevenuePerRental	HighestSingleRental	LowestSingleRental
1 April	2026	2	16000.00	8000.000000	12000.00	4000.00

ReportMonth	ReportYear	TotalReservation	TotalDepositRevenue	AverageRevenuePerReservation
1 April	2026	2	12000.00	6000.000000

ReportMonth	ReportYear	TotalRevenue
1 April	2026	28000.00

Payment Table

PaymentID	PaymentDate	PaymentMethod	PaymentStatus	PaymentType	AmountPaid	RentalID	ReservelD
1 P01	2026-04-09	Card	Completed	Deposit	5000.00	NULL	R001
2 P02	2026-04-09	Card	Completed	Deposit	7000.00	NULL	R002
3 P03	2026-04-09	Card	Completed	Rental	4000.00	rid001	NULL
4 P04	2026-04-09	Cash	Completed	Rental	12000.00	rid002	NULL

9. Using sp_check_vehicle_availability

Testing Script

```
--CHECKING VEHICLE STATUS WITH sp_check_vehicle_availability
--Check 'V001'
EXEC sp_check_vehicle_availability 'V001' ;
--Check 'V002'
EXEC sp_check_vehicle_availability 'V002' ;
--Check 'V003'
EXEC sp_check_vehicle_availability 'V003' ;
--Check 'V004'
EXEC sp_check_vehicle_availability 'V004' ;
--Check 'V005'
EXEC sp_check_vehicle_availability 'V005' ;
```

Result

Before Converting to Rental

Vehicle	Status
Vehicle V001	is Available
Vehicle V002	is Available
Vehicle V003	is Available
Vehicle V004	is Available
Vehicle V005	is Available

After Converting to Rental

Vehicle	Status
Vehicle V001	is Rented
Vehicle V002	is Rented
Vehicle V003	is Available
Vehicle V004	is Available
Vehicle V005	is Available

After returning vehicle and making payment

Vehicle	Status
Vehicle V001	is Available
Vehicle V002	is Available
Vehicle V003	is Available
Vehicle V004	is Available
Vehicle V005	is Available

Note: `tg_update_vehicle_status` is also tested and correctly executed. The vehicle status is **Available** as the rental status is **Completed**; hence the trigger is successful.

10. Cursors

Overdue_rental_cursor

Testing Script

```
DECLARE @RentalID VARCHAR(10);

DECLARE Overdue_rental_cursor CURSOR FOR
SELECT RentalID
FROM Rental
WHERE ActualReturnDate > RentalEndDate;

OPEN Overdue_rental_cursor;
FETCH NEXT FROM Overdue_rental_cursor INTO @RentalID;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Apply late fee using your stored procedure
    EXEC sp_ApplyLateFee @RentalID;

    FETCH NEXT FROM Overdue_rental_cursor INTO @RentalID;
END
CLOSE Overdue_rental_cursor;
DEALLOCATE Overdue_rental_cursor;
```

Result

Messages
Commands completed successfully.

Update_vehicle_status_cursor

Testing Script

```
DECLARE @vehicleID CHAR(10)

DECLARE Update_vehicle_status_cursor CURSOR FOR
SELECT VehicleID
FROM Rental
WHERE ActualReturnDate IS NOT NULL -- Vehicle that has been returned

OPEN Update_vehicle_status_cursor
FETCH NEXT FROM Update_vehicle_status_cursor INTO @vehicleID
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE Vehicle
    SET VehicleStatus = 'Available'
    WHERE VehicleID=@vehicleID

    FETCH NEXT FROM Update_vehicle_status_cursor INTO @vehicleID;
END
CLOSE Update_vehicle_status_cursor
DEALLOCATE Update_vehicle_status_cursor
```

Result

Messages
Commands completed successfully.

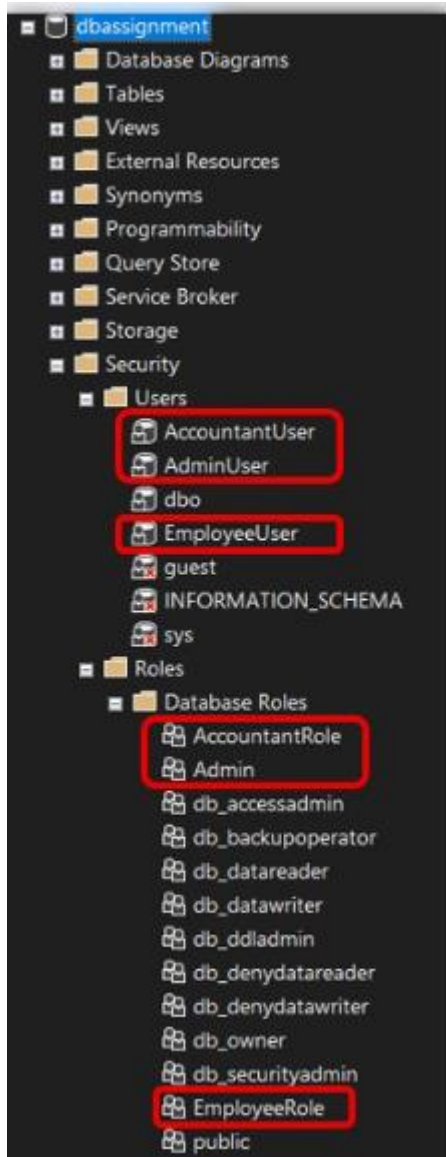
11. Security

Testing Script

Refer to Code File in the Security Section. Note that all commands have been executed at the same time (in batch).

Result

Messages
Commands completed successfully.



The **users** and **roles** for the Car Rental Database System have been defined in Microsoft SQL Server. The system includes three primary user roles: **Admin**, **Employee**, and **Accountant**.

References

- [1] Connolly, T. and Begg, C. (2005) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 4th ed. Harlow: Addison-Wesley.
- [2] Microsoft Docs (2024) *SQL Server Documentation*. Available at: <https://docs.microsoft.com/en-us/sql/> (Accessed: 13 March 2026).
- [3] Studocu (2022/2023) *CS Database Management System Project: ER Diagram for Car Rental System*. University of Southern California. Available at: <https://www.studocu.com/en-us/document/university-of-southern-california/database-systems/dbms-er-diagram/42138313> (Accessed: 28 February 2026).

Note: The submitted code file contains all SQL scripts referenced in this report, including table creation statements, stored procedures, triggers, cursors, and queries. The testing section is also included at the end of the file.

Code file: Car_Rental_2510502.sql

Individual Contribution

Name	Student ID	Sections Worked	Percentage(%)
Govind Dhruv	2510502	Core Functional Modules Entity Relationship Diagram (ERD) Employee table sp_calculate_rental_cost tg_update_payment_status Overdue_rental_cursor Employee role + grant permissions for Employee + Assigning Users to Roles Customer Rental History Query	24%
Sahajoo Ronitash	2512131	Scope & Limitations Payment + Maintenance table sp_apply_late_fee tg_auto_late_fee Deny permissions	15%
Khedoo Muhummad Younus Khan Bin Yousouf	2513292	Detailed System Description Normalisation Customer table sp_add_vehicle, sp_check_vehicle_availability tg_overlapping_reservation Update_vehicle_status_cursor User logins and passwords Admin role + grant permissions for Admin Late return list Query	23%
Makhun Lokeshav Anand	2514204	Business Rules VehicleCategory + Vehicle table sp_make_reservation, sp_monthly_revenue_report sp_make_payment, sp_vehicle_returned Accountant role + grant permissions for Accountant Insert queries + Testing Section	23%
Bungchee Tejasvir	2514012	Users of the System Reservation + Rental table sp_add_customer, sp_convert_reservation_to_rental tg_update_vehicle_status Most Rented Vehicle Query	15%